ЛЕКЦИЯ 5

Graph Transformers и масштабирование

От устройства модели к Graphormer, GraphGPS и масштабированию

Графовые нейронные сети

Задача: для ответа в A нужен признак далёкой E

- Строим прогноз для вершины A
- В учебной задаче полезный признак находится в E
- Связь A–E отсутствует: между ними путь через другие вершины



Нужен прогноз

Нужен признак E

Как доставить полезную информацию из E в A?

Локальной GNN нужны слои, чтобы пройти путь

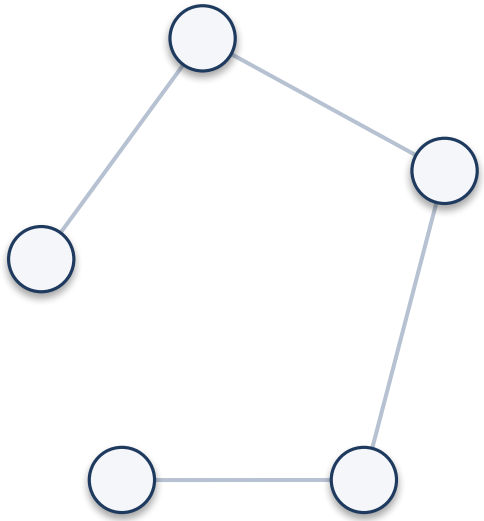
- Один слой передаёт сообщения по одному ребру
- После двух слоёв A зависит от признаков A, B и C
- Признак E сможет дойти до A через 4 локальных слоя



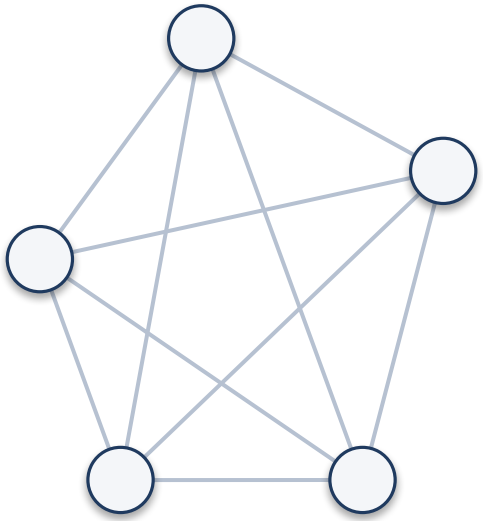
Больше слоёв расширяет область доступа и добавляет вычисления.

Что нового после GAT?

- В GAT вершина выбирала, каких соседей слушать
- Теперь она может напрямую получить информацию от далёкой вершины
- При этом сообщаем модели, как вершины связаны в исходном графе



GAT: слушаем соседей

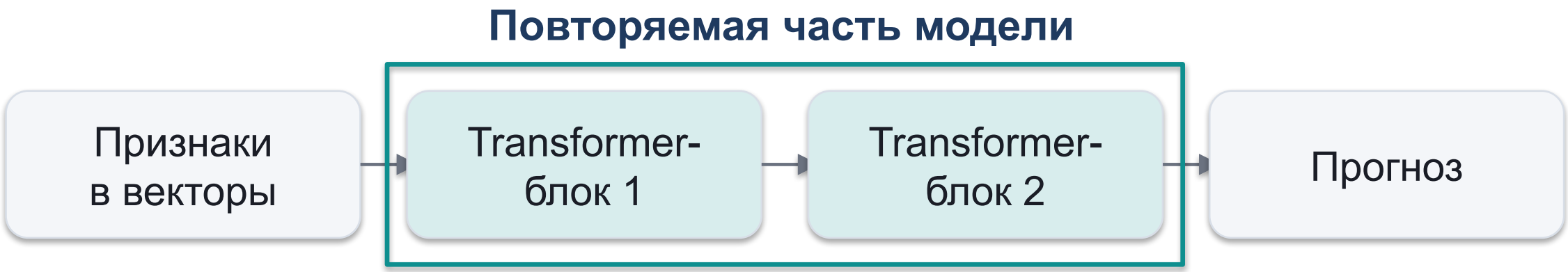


Теперь: доступны все вершины

Внимание и несколько голов уже были в GAT.

Дальше учимся делать такой обмен на большом графе и укладываться в память.

Вся модель: признаки, блоки, прогноз

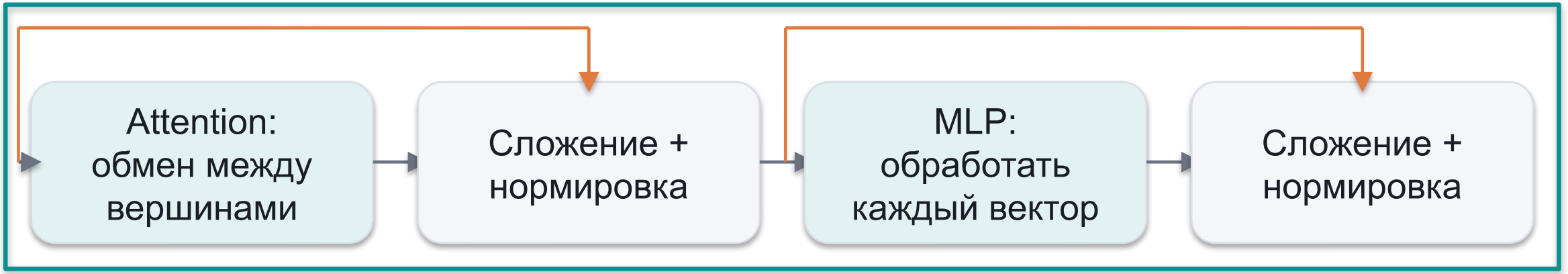


Transformer — название архитектуры. Блок — одна повторяемая часть этой архитектуры.

У блоков одна схема, но каждый учит свои параметры. На выходе блока — новые векторы.

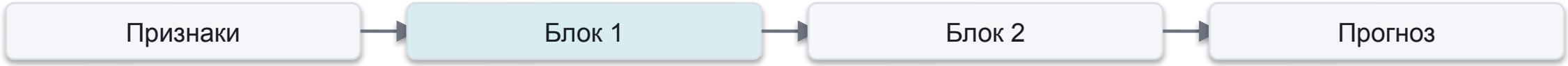
РАСКРЫВАЕМ БЛОК 1 ИЗ ОБЩЕЙ СХЕМЫ

Один Transformer-блок крупным планом



Весь обведённый фрагмент — один блок. Attention — его первая часть.

MLP выполняется один раз. Оранжевые линии — входы, сохранённые для сложения.



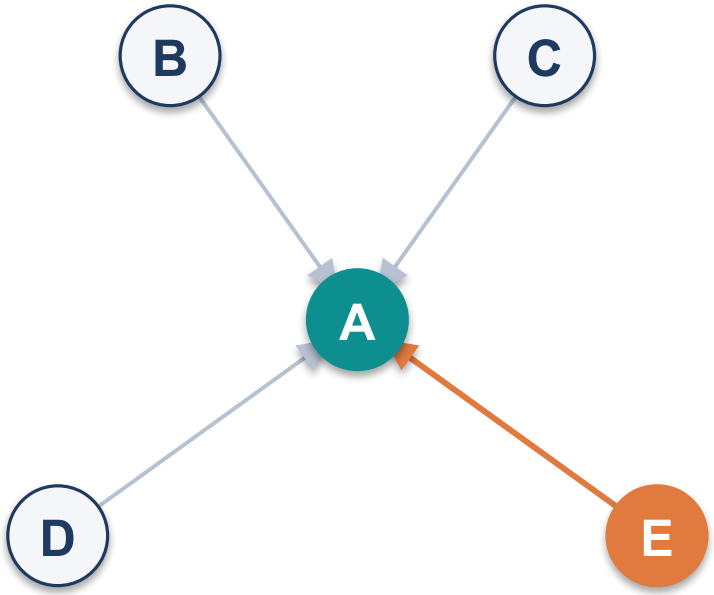
Transformer в тексте, изображениях и графах

Область	Что является элементом?	Какую структуру учитываем?
Текст	Токен: слово или часть слова	Порядок токенов в тексте
Изображение	Патч: небольшой фрагмент картинки	Расположение фрагментов
Граф	Вершина: например, атом или статья	Связи между вершинами

Общий механизм: attention и MLP. Входы, маски и выходные задачи могут различаться.

Метод self-attention: взвесить доступные источники

- Вершина собирает новое представление из доступных источников, включая себя
- Модель вычисляет вес каждого доступного источника для данного получателя
- В глобальном варианте A может использовать E в этом же слое



Self: получатели и источники — вершины одного и того же графа.

H: текущие признаки всех вершин

Вершина	Строка h_i в матрице H
1	[1, 0, 2]
2	[0, 1, 4]
3	[2, 2, 8]

H: N строк по числу вершин, d столбцов по числу признаков.

h_1 — вектор одной вершины. H — все такие векторы вместе.

Для сравнения и передачи разделим признаки: Q, K и V

Q (query): получатель

K (key): сравнение источников

V (value): передаваемые признаки

Все три набора получаем из H: учим разные линейные преобразования одних признаков.

ТЕ ЖЕ H И ВЕСА ИСПОЛЬЗУЕМ ДЛЯ ВСЕХ ТРЁХ ВЕРШИН

Пример: как из h_1 получить q_1, k_1 и v_1?

Преобразование	Выбранный столбец W	Расчёт для $h_1 = [1, 0, 2]$
Query	$W_Q = [1, 0, 0]^T$	$q_1 = 1\times1 + 0\times0 + 2\times0 = 1$
Key	$W_K = [0, 1, 0]^T$	$k_1 = 1\times0 + 0\times1 + 2\times0 = 0$
Value	$W_V = [0, 0, 1]^T$	$v_1 = 1\times0 + 0\times0 + 2\times1 = 2$

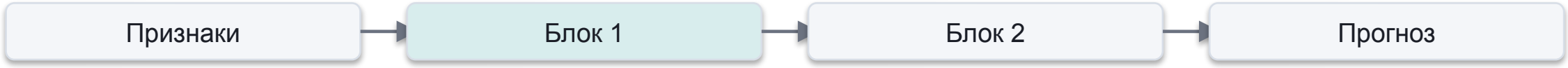
Здесь каждая роль задаётся одним числом. В модели матрицы W обучаются.

Attention на числах: считаем только вершину 1

Источник	Ключ	Оценка	Вес	Значение	Вклад
1	0	1 × 0 = 0	0.090	2	0.180
2	1	1 × 1 = 1	0.245	4	0.979
3	2	1 × 2 = 2	0.665	8	5.322

Softmax превращает оценки в положительные веса с суммой 1.

Новый вектор вершины 1: 0.180 + 0.979 + 5.322 ≈ 6.481.



ПОЛУЧАТЕЛЬ 1; BIAS ВЫБРАН ВРУЧНУЮ ДЛЯ ПРИМЕРА

Маска и bias действуют до softmax

Вариант	Оценки перед softmax	Смысл
Dense	[0, 1, 2]	Все источники разрешены
Local mask	[0, 1, −∞]	Источник 3 исключён
Прибавили distance bias	[0, 0.5, 1]	Меньше вес вершины 3

Исходные оценки [0, 1, 2] + bias [0, −0.5, −1] = итог [0, 0.5, 1]

Веса после softmax: [0.186, 0.307, 0.506]. Все источники участвуют.

Self-attention и multi-head — два разных свойства

Термин	На какой вопрос отвечает?	В нашем примере
Self-attention	Откуда берутся запросы и источники?	Q, K и V получены из одного H
Multi-head	Сколько параллельных наборов весов внимания?	У каждой головы свои проекции и веса источников

Результаты голов объединяем и возвращаем исходную длину вектора.

Multi-head self-attention сочетает оба свойства. GAT тоже может быть multi-head.

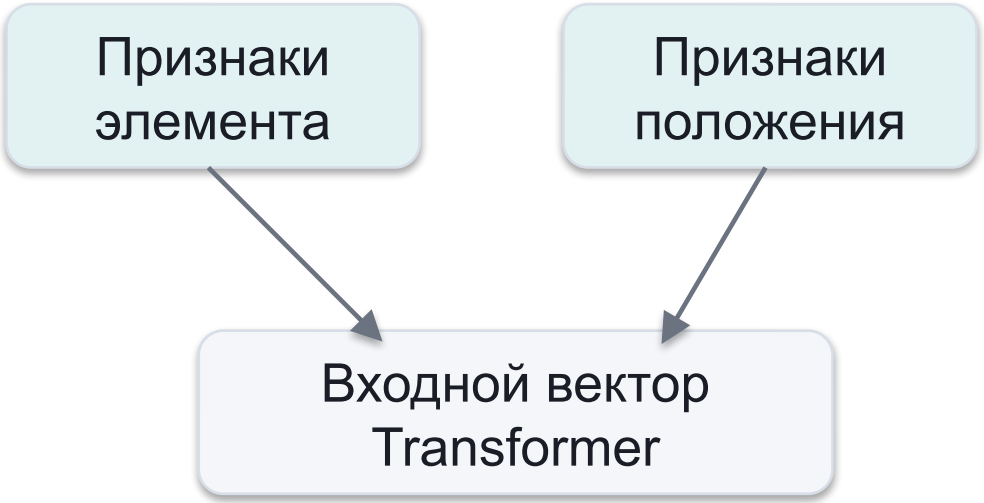
GAT и глобальный attention: общее и различия

Свойство	Исходный GAT из лекции 3	Глобальный dot-product attention
Доступные источники	Соседи по рёбрам + сама вершина	Все вершины одного графа
Как оценивается пара?	Склейка двух векторов и обучаемая оценка	Скалярное произведение query и key
Несколько голов	Есть такой вариант	Есть такой вариант
Где задаётся граф?	Рёбра ограничивают источники	Добавляем признаки структуры и bias пар

Оба метода используют softmax и взвешенную сумму признаков источников.

PE: добавляем сведения о положении элемента

- PE = positional encoding, позиционные признаки
- В тексте важно место токена, в картинке — место фрагмента
- В графе положение описываем через связи; произвольный ID вершины его не задаёт



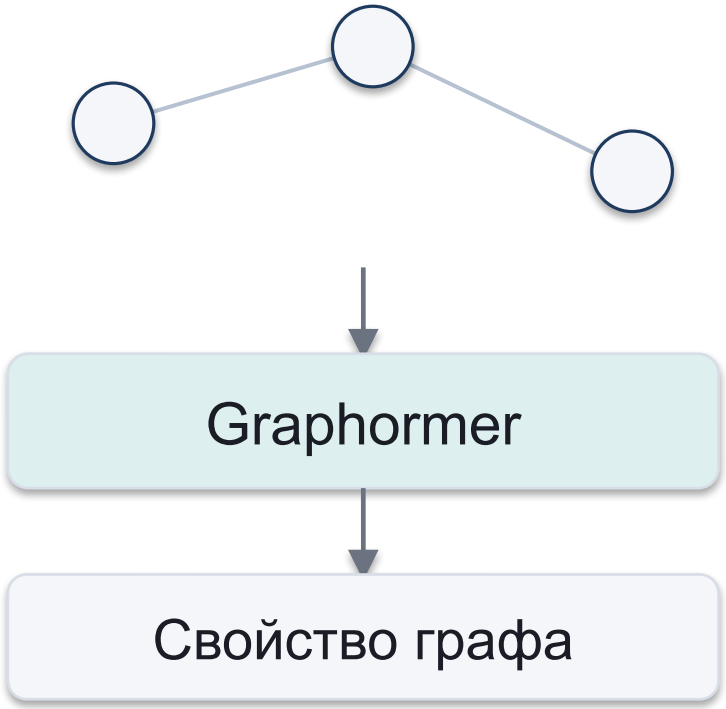
На графах один из вариантов PE — LapPE, признаки из спектра графа.

RWSE — пример структурных признаков: вероятности возврата случайного блуждания.

YING ET AL.; MICROSOFT RESEARCH И СОАВТОРЫ; NEURIPS 2021

Graphormer: Transformer для графовых данных

- Graphormer адаптирует Transformer к графам через явные сведения об их структуре
- На входе — вершины, их признаки и связи между ними
- Основная задача исходной работы — предсказывать свойства целого графа, например молекулы



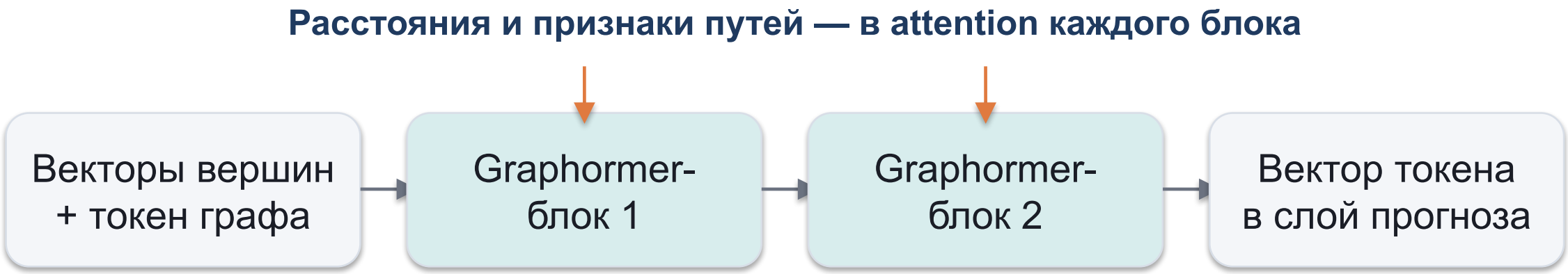
Здесь разбираем исходную архитектуру Graphormer 2021 года.

Как Graphormer получает структуру графа?

Механизм	Какую информацию даёт?	Куда её добавляем?
Centrality encoding	Степень: число соседей вершины	Обучаемый вектор — к признакам вершины
Spatial encoding	Кратчайшее расстояние между парой вершин	Обучаемое число — к оценке внимания этой пары
Edge encoding	Признаки рёбер кратчайшего пути, если они есть	Ещё одну обучаемую поправку — к оценке пары

Attention сравнивает признаки вершин с учётом их положения и связей в графе.

Graphormer целиком: от графа до прогноза



В каждом блоке — attention с графовыми поправками и MLP.

Токен графа — служебный вектор: обновляется вместе с вершинами и представляет весь граф.

Graphormer: три входа структуры на цепочке

- Степень — число соседей. У вершин 1 и 3 по одному соседу, у вершины 2 — два
- Расстояние — число рёбер на пути. От 1 до 3 нужно пройти 2 ребра
- Тип связи — свойство ребра. В молекуле это, например, одинарная или двойная химическая связь



Двойная связь ведёт к одному соседу. Типы связей берём из данных, если они есть.

Степень кодируем в признаках; расстояние и типы связей меняют оценки пар.

RAMPÁŠEK ET AL., NEURIPS 2022

GraphGPS: объединяем локальную GNN и Transformer

- GraphGPS — рецепт сборки графового Transformer из независимых модулей
- Локальная GNN обрабатывает реальные рёбра; глобальное внимание связывает далёкие вершины
- Выбираем кодировки, GNN и вид внимания под задачу и размер графа

Выбираем три компонента

Признаки положения и структуры

Локальная GNN

Глобальное внимание

GPS: General, Powerful, Scalable Graph Transformer.

В практике используем GPSConv — готовый GPS-слой из PyTorch Geometric.

Graphormer и GraphGPS: два варианта графовой модели

Свойство	Graphormer, который мы разобрали	GraphGPS
Как устроен обмен?	Глобальный attention с графовыми поправками	Локальная GNN и глобальный attention параллельно
Где используются признаки рёбер?	В поправках по кратчайшим путям	В сообщениях локальной GNN
Что выбираем при сборке?	Рассмотрели конкретную архитектуру 2021 года	Кодировки, тип GNN и глобальный модуль

Новое после Graphormer: возвращаем локальную GNN как отдельную ветвь и выбираем модули.

ПРИ ФИКСИРОВАННОЙ ШИРИНЕ МОДЕЛИ И НАСТРОЙКАХ АППРОКСИМАЦИИ

Стоимость GraphGPS зависит от глобального модуля

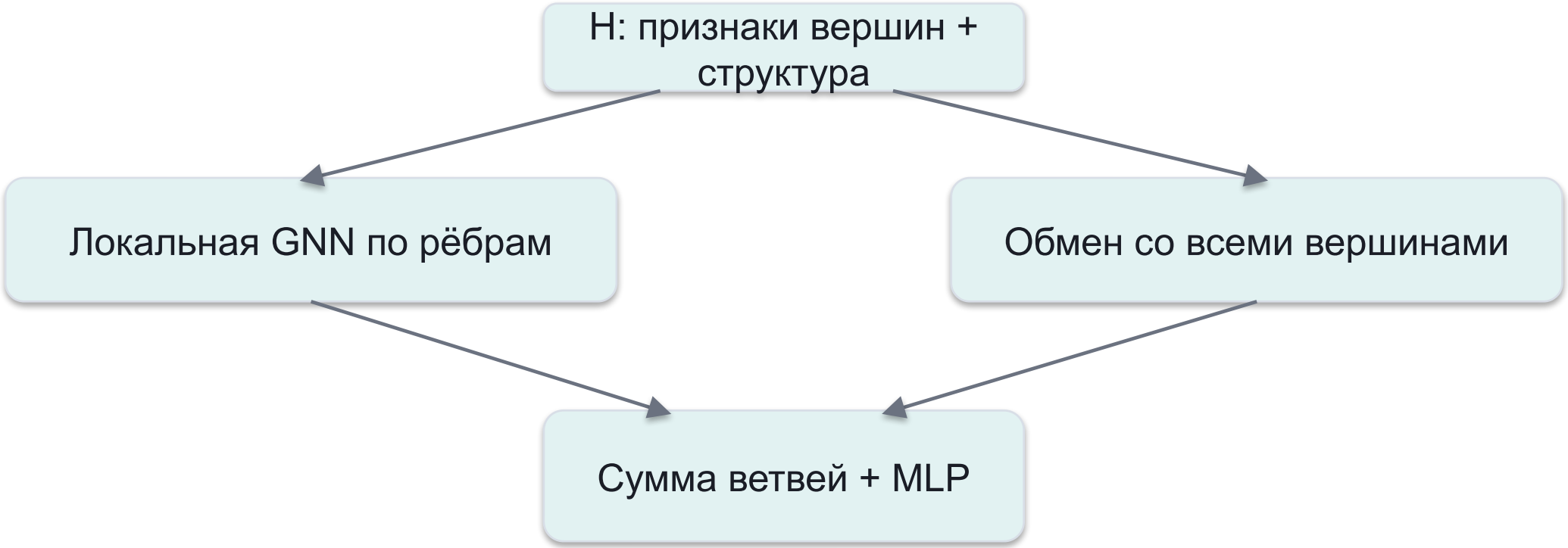
Глобальный модуль	Как передаёт информацию?	Цена GPS-слоя
Полное внимание	Сравнивает все пары вершин	$O(E + N^2)$
Performer	Приближённо использует общую сводку источников	$O(N + E)$

N — число вершин, E — рёбер. Предобработка кодировок считается отдельно.

В практике выбран полный attention: глобальная ветвь здесь остаётся квадратичной.

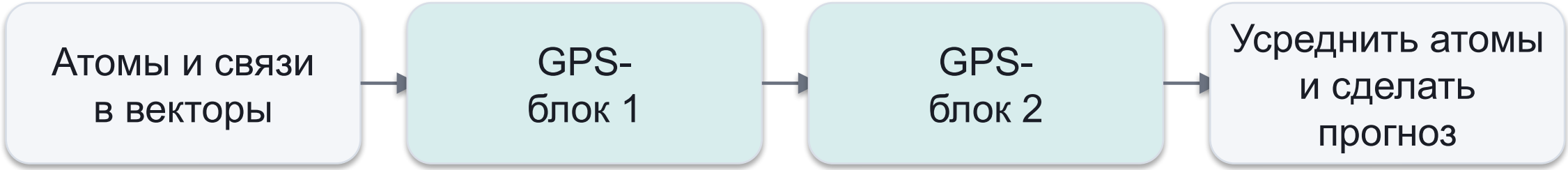
RAMPÁŠEK ET AL., 2022

GraphGPS: графовый блок в той же модели



Здесь обычный блок заменяем GPS-блоком: локальный обмен + глобальное attention.

Модель практики: от молекулы до одного числа

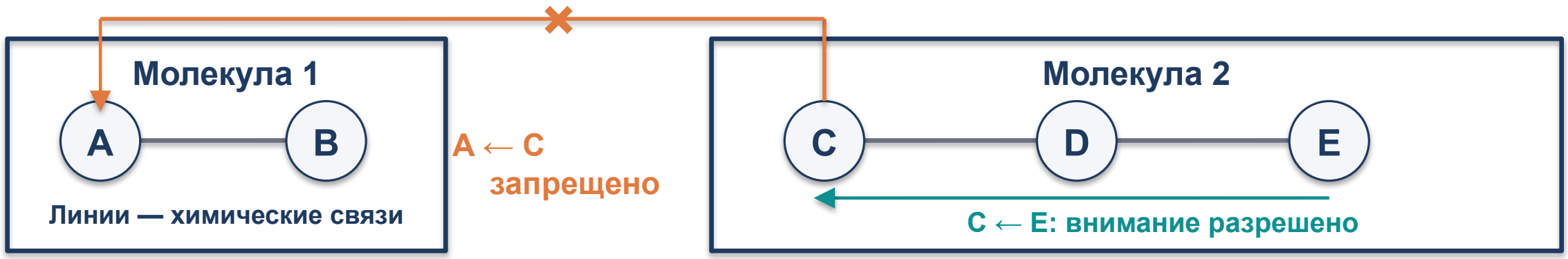


Каждый GPS-блок обновляет все векторы атомов. Итоговое число выдаёт слой прогноза.

Типы атомов превращаем в векторы; типы связей использует локальная ветвь.

A–E — МЕТКИ АТОМОВ В УЧЕБНОЙ СХЕМЕ, НЕ ХИМИЧЕСКИЕ СИМВОЛЫ

Две молекулы: внимание внутри каждой



	A	B	C	D	E
A	1	1	0	0	0
B	1	1	0	0	0
C	0	0	1	1	1
D	0	0	1	1	1
E	0	0	1	1	1

Строка получает, столбец передаёт

Можно: C получает от E внутри одной молекулы, даже без прямой химической связи.

Нельзя: A получает от C из другой молекулы. Прогнозы двух молекул должны быть независимы.

1 — можно, 0 — нельзя. Всего $2^2 + 3^2 = 13$ пар. Считаем оба направления и внимание к себе.

ОЦЕНКА: 8 ГОЛОВ, 4 БАЙТА НА КАЖДУЮ ОЦЕНКУ ПАРЫ

Сколько памяти занимает таблица внимания?

Вершин N	Пар N^2	Память, ГиБ
1,000	1,000,000	0.03
10,000	100,000,000	2.98
169,343	28,677,051,649	854.64

Это одна таблица оценок пар, без остальных тензоров модели.

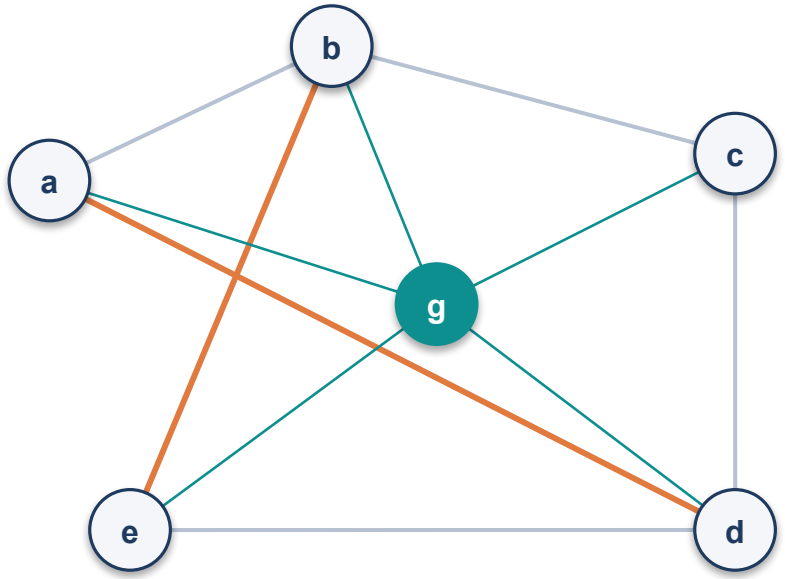
Специальные реализации экономят её хранение; число пар всё равно N^2.

Что уменьшаем при масштабировании?

Проблема	Приём	Компромисс
Слишком много пар attention	Разреженные связи или общая сводка	Меняем доступные пары или приближаем attention
Слишком большая окрестность GNN	Выбор соседей или подграфа	Часть контекста пропускаем
Слишком много повторных вычислений	Кеш представлений	Нужна память; значения могут устаревать

Sparse attention: дополнительные связи вместо всех пар

- Сохраняем исходные рёбра графа
- Добавляем разреженные дальние связи
- Общая вершина g собирает и раздаёт информацию



Ограничиваем число добавленных связей на вершину: O(E + N) пар.

Масштабирование: уменьшаем объём обмена

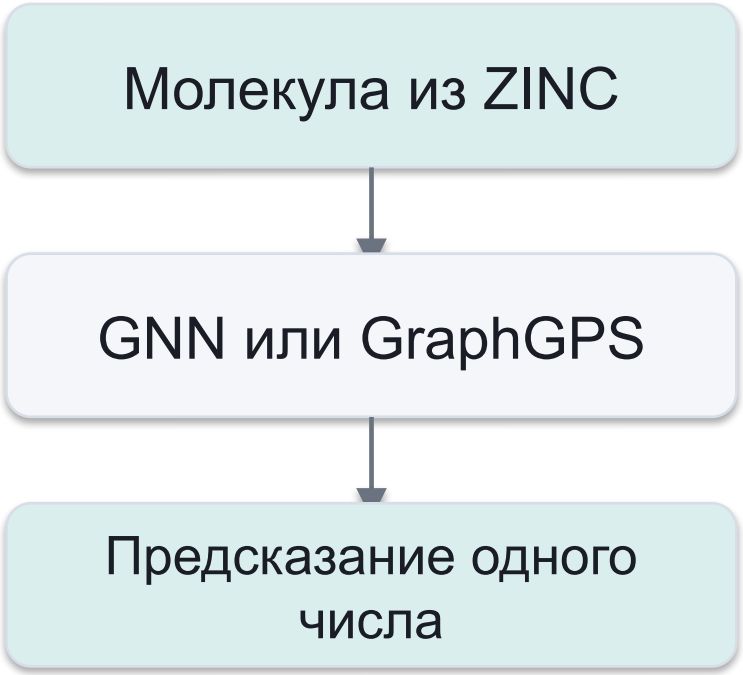
Attention	Локальная GNN	Что проверяем?
Разрешаем меньше пар или приближённо считаем внимание	**Берём целевые вершины и часть их соседей**	**Помещается ли в память? Сколько занимает обучение? Как меняется ошибка?**

В нашей практике: небольшие молекулы и полный attention, без выборки соседей.

Выборка соседей называется sampling. Уменьшаем затраты и проверяем качество.

ZINC — набор данных молекул

- Один пример — одна молекула: атомы становятся вершинами, химические связи — рёбрами
- Для каждой молекулы дано целевое число y; модель учится предсказывать его по графу
- В практике сравниваем локальную GNN и GraphGPS на одних и тех же молекулах



Цель здесь — penalized logP: составная химическая оценка молекулы.

Практика 5: данные, обучение, сравнение

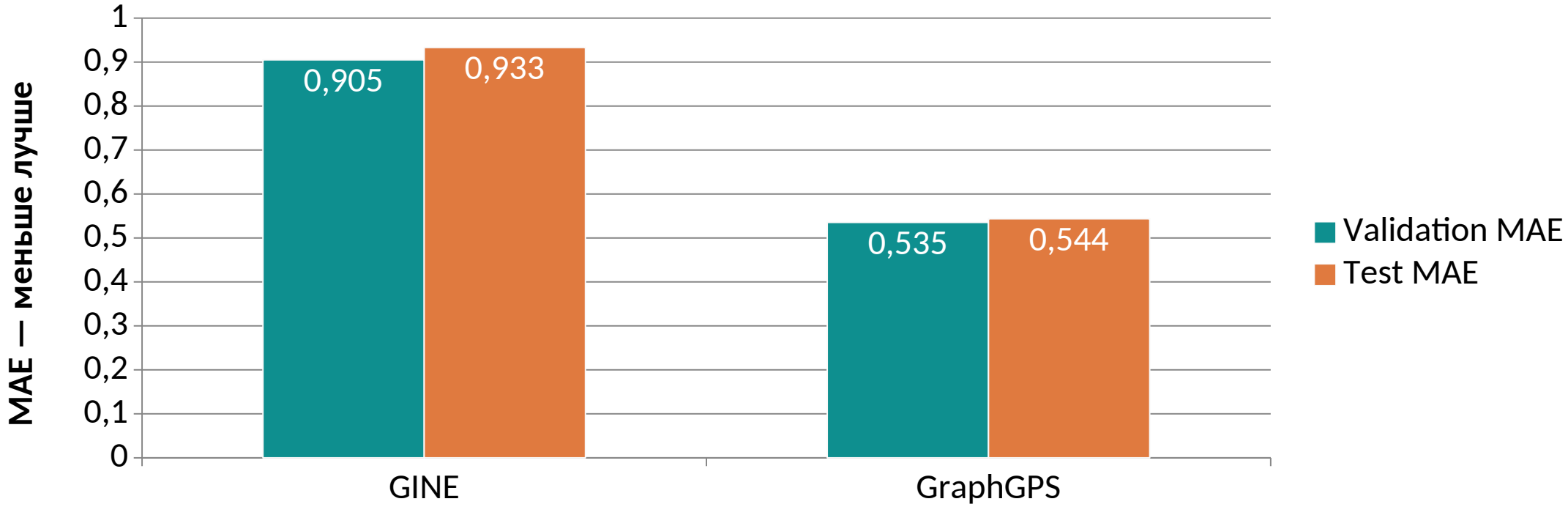
Шаг	Что делаем?	Что получаем?
Данные ZINC	Смотрим атомы, связи и цель y	Понимаем один пример и разбиения
Готовые слои	Обучаем GINE и GPSConv: 10 эпох, 3 seed	Две конфигурации на одинаковых данных
Оценка	Считаем MAE на test, разброс и время	Какая модель лучше и на сколько

GINE — локальная GNN: суммирует сообщения соседей с учётом признаков связей.

В нашей GraphGPS эта же GINE — локальная ветвь, рядом работает глобальное внимание.

512/128/128 МОЛЕКУЛ · 10 ЭПОХ · СРЕДНЕЕ ПО 3 SEED

ZINC: какая модель показала себя лучше?



GraphGPS: test MAE ниже на 0.389 (41.7%). Результат учебного опыта.

Главная схема, к которой возвращаемся

1. Вся модель: входные векторы, несколько блоков, слой прогноза
2. Один блок: attention, прибавление входа и нормировка, MLP, прибавление входа и нормировка
3. Graphormer добавляет структуру в признаки и веса; GPS объединяет локальную и глобальную ветви
4. Для большого графа ограничиваем пары attention, контекст GNN или повторные вычисления

Литература и инструменты

- **Vaswani et al., 2017** — Transformer · arxiv.org/abs/1706.03762
- **Ying et al., 2021** — Graphormer · arxiv.org/abs/2106.05234
- **Rampášek et al., 2022** — GraphGPS · arxiv.org/abs/2205.12454
- **Shirzad et al., 2023** — Exphormer · arxiv.org/abs/2303.06147
- **Choromanski et al., 2021** — Performer · arxiv.org/abs/2009.14794
- **PyG documentation** — NeighborLoader и graph_transformer tutorials